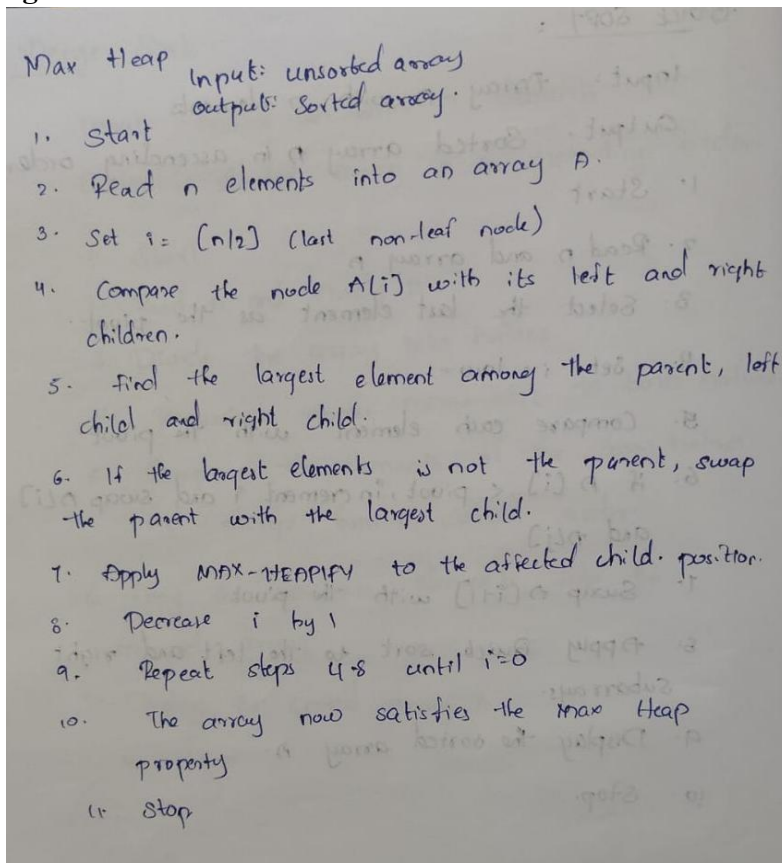


## Practical – 3

**Aim :** To Implementation of Max-Heap sort algorithm

**Algorithm :**



Max heap      Input: unsorted array  
                                Output: Sorted array.

1. Start
2. Read  $n$  elements into an array  $A$ .
3. Set  $i = \lfloor n/2 \rfloor$  (last non-leaf node)
4. Compare the node  $A[i]$  with its left and right children.
5. Find the largest element among the parent, left child and right child.
6. If the largest element is not the parent, swap the parent with the largest child.
7. Apply MAX-HEAPIFY to the affected child position.
8. Decrease  $i$  by 1
9. Repeat steps 4-8 until  $i=0$
10. The array now satisfies the max heap property
11. Stop

**Program / Code :**

```
#include <iostream>
#include <vector>
using namespace std;

void heapify(vector<int> &arr, int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapsort(vector<int> &arr)
{
    int n =
    static_cast<int>(arr.size());

    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90};

    heapsort(arr);
}
```

```
cout << "Sorted array using heap sort:\n";  
for (int value : arr)  
cout << value << ' ';  
  
cout << '\n';  
return 0;  
}
```

**OUTPUT:**

```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\heapsort.exe  
Sorted array using heap sort:  
11 12 22 25 34 64 90
```

**Time Complexity :**

Time Complexity:

for ( $i = n/2, i > 1; i--$ )  $\rightarrow n$   
 max heapify ( $arr, n, i$ )  $\rightarrow \log n$   
 for ( $i = n; i > 1; i--$ )  $\rightarrow n$   
 swap ( $arr[i], arr[1]$ )  $\rightarrow 1$   
 max heapify ( $arr, i-1, 1$ )  $\rightarrow \log n$

$$T(n) = n \times \log n + n + 1 \times \log n$$

$$= n \log n + n \log n$$

$$T(n) = O(n \log n)$$

i) Best case:

for  $i = n/2$  to 1  
 max heapify ( $arr, n, i$ )  $\rightarrow n$   
 for  $i = n$  to 2  
 swap heapify ( $arr, i-1, i$ )  $\rightarrow \log n$

$$T(n) = O(n) + n \times O(\log n)$$

$$= O(n \log n)$$

ii) Average case:

for  $i = n/2$  to 1  
 max heapify ( $arr, n, i$ )  $\rightarrow n$   
 for  $i = n$  to 2  
 swap ( $arr[i], arr[1]$ )  
 max heapify ( $arr, i-1, 1$ )  $\rightarrow \log n$   
 $T(n) = n + \log n$   
 $= O(n \log n)$

iii) Worst case:

max heapify from root to lowest level  
 $\rightarrow \log n$   
 max heapify  $\rightarrow \log n$   
 $T(n) = n \times \log(n) \rightarrow O(n \log n)$

**Conclusion :**

The Max-Heap Sort algorithm was successfully implemented and executed to sort the elements in ascending order. The program verified a consistent time complexity of  $O(n \log n)$  across the best, average, and worst cases using an in-place binary heap structure